Diseño y Verificación en VHDL de un Datapath de 8 Bits: Datapath 1 — MUX, ALU, Registro Realimentado y Buffer de Salida

Aldo Mauricio Villegas Peralta, Uriel Lemus Pérez, Max Ramos Martínez, Alfredo Ramírez Candia, Santiago Silverio Flores

Equipo de Arquitectura de Computadoras,
Benemérita Universidad Autónoma de Puebla, Puebla, México
{vp202368617, lp202362797, rm202376182, rc202375947, sf202367190}@alm.buap.mx

Resumen. Se diseña y verifica por simulación un datapath de 8 bits con MUX de entrada, ALU de tres operaciones ($n + x$, $2n + x$, $n + 1$), registro con carga realimentado a la ALU y buffer tri-estado de salida. El circuito se describe en VHDL-2008 y se valida con cinco bancos de prueba autochequeados en GHDL, analizando las ondas con GTKWave. Bajo el modelo especificado, las operaciones, la carga, la retención con $L = 0$ y la alta impedancia se comportan como se esperaba; la aritmética es módulo 2^8 sin banderas.

Palabras Clave: Arquitectura de Computadoras, Datapath, VHDL, Simulación, GHDL

1 Introducción

Para afrontar el diseño de sistemas digitales, la ingeniería recurre a abstracciones por capas: quien escribe código ve instrucciones y no transistores, y quien diseña la unidad de procesamiento la compone a partir de bloques funcionales delimitados [14]. En esa descomposición, la unidad central de procesamiento (CPU) se separa convencionalmente en una ruta de datos o *datapath*, encargada de transportar y transformar los datos, y una unidad de control, que secuencia las operaciones [6]. Las implementaciones de referencia de ciclo único ilustran exactamente esta separación: un datapath formado por MUX, ALU y registros, gobernado por una unidad de control que genera una palabra de control por ciclo [14,3]. El crecimiento sostenido del número de transistores [8] ha desplazado buena parte del esfuerzo de diseño hacia la descripción y la verificación automatizadas: antes de construir, el circuito se modela en un lenguaje de descripción de hardware y se valida por simulación. En el marco del curso de Arquitectura de Computadoras, este trabajo toma el circuito de datapath propuesto en el material de clases —cuyo esquema muestra un MUX de entrada, una ALU con tres operaciones, un registro con carga y un buffer de salida con habilitación—

y lo desarrolla como un modelo VHDL-2008 completo [10] verificado con bancos de prueba autochequeados en GHDL [4] y analizado con GTKWave [5].

El datapath resultante es una ruta de datos de 8 bits, parametrizable por medio de un genérico, con cuatro bloques: un MUX $2 \to 1$ que elige entre un valor externo y la constante cero; una ALU que implementa tres operaciones sobre sus dos operandos ($n + x$, $2n + x$ y $n + 1$); un registro síncrono con carga y reinicio; y un buffer tri-estado que conecta la salida externa. La secuencia no la genera ninguna máquina de estados: una palabra de control de seis bits distribuidos en cuatro campos, aplicada directamente por el entorno de simulación, distribuye el bit de habilitación de salida, la carga del registro, la selección del MUX y el código de operación de la ALU. Ese alcance es deliberado: sin unidad de control ni autómata finito [12], se modela y verifica únicamente la ruta de datos, que es el componente ejecutor del computador [6].

Los objetivos del trabajo son: (i) describir en VHDL-2008 los módulos del datapath (MUX, ALU, registro y buffer) con ancho parametrizable; (ii) integrarlos en una entidad estructural con la realimentación del registro hacia la ALU y el desempaquetado de la palabra de control; (iii) verificar cada módulo y el conjunto con bancos de prueba autochequeados; (iv) analizar el comportamiento temporal observado en las ondas de simulación; y (v) documentar el formato de la palabra de control, las operaciones de la ALU y los límites aritméticos del diseño.

El artículo se organiza como sigue. La Sección 2 presenta el marco teórico, la arquitectura y los módulos VHDL. La Sección 3 describe la propuesta experimental y las herramientas. La Sección 4 muestra las simulaciones, la Sección 5 analiza los resultados y la Sección 6 recoge las conclusiones y el trabajo futuro.

2 Diseño Teórico

2.1 Marco teórico

El datapath es el bloque de la CPU que ejecuta las operaciones sobre los datos: mueve operandos, los transforma y almacena resultados. Por sí solo no decide qué hacer: recibe de la unidad de control una serie de señales que configuran sus MUX, su ALU y sus registros ciclo a ciclo [6,14]. En los diseños de libro esa unidad de control se modela habitualmente como una máquina de estados de estilo Moore o Mealy [12,7,2,16]; formalmente, un autómata finito es una quíntupla de estados, entradas, función de transición, estado inicial y estados finales [16], y las señales de control se reparten en campos codificados dentro de una microinstrucción [11,18].

Este trabajo prescinde de esa unidad de control, por criterio del alcance del encargo: se modela únicamente la ruta de datos y la palabra de control se aplica desde el banco de pruebas. La consecuencia metodológica es que cada microoperación se define como una palabra concreta del puerto `CW`, lo que hace el comportamiento del circuito completamente determinista y directamente verificable sin necesidad de programar un secuenciador.

En el lado secuencial, el único elemento de memoria es un registro con habilitación de carga que trabaja de forma síncrona [12]; su modelado recurre a los procesos y a las sentencias condicionales y de caso del lenguaje [1]. La salida del conjunto se habilita mediante un buffer tri-estado, que deja el bus en alta impedancia (‘`Z`’) cuando no está activo [18]; los nueve valores de la lógica resoluble que incluyen esa alta impedancia están definidos por el paquete `std_logic_1164` del estándar [9].

Por su parte, la ALU combina operaciones aritméticas y de corrimiento dentro de un mismo bloque [15]. Existen estructuras dedicadas, los *barrel shifters*, que desplazan N posiciones en un solo ciclo [11], pero el circuito de origen no incluye un shifter independiente: la única operación de corrimiento necesaria —el desplazamiento de un bit que produce el doble de n— se implementa dentro de la propia ALU (§2.4).

2.2 Arquitectura general

La Fig. 1 muestra la arquitectura propuesta. El valor externo `dato_ext` y la constante cero compiten en un MUX $2 \to 1$ controlado por el campo `M`; su salida es el operando A de la ALU. El operando B proviene de la realimentación: es la salida Q del registro, que por eso almacena el valor n. La ALU, gobernada por el campo `C`, produce el resultado combinacional F_{ALU}, que entra como dato D del registro. Este se actualiza en el flanco de subida de `clk` cuando la habilitación `L` lo permite, y su salida Q—además de realimentar B—atraviesa un buffer tri-estado habilitado por `O.E` hasta la salida externa. Los pulsos `clk` y `rst` alimentan al registro y son decisión de diseño: la fotografía del material de clase no etiqueta un reloj.

Las cuatro señales que intervienen son distintas y conviene no confundirlas: F_{ALU} es el resultado *combinacional* de la ALU; D es la entrada del registro (en este diseño, el mismo cable que F_{ALU}); Q es el valor *secuencial* almacenado, que cambia únicamente en el flanco de reloj; y `salida` es el valor de Q visible cuando `O.E` $= 1$. En forma compacta:

$$
\begin{aligned}
&A = \texttt{M}\,?\,\texttt{dato_ext} : 0, \qquad && \texttt{salida} = \texttt{O.E}\,?\,Q : \texttt{Z},\\
&B = Q, && F_{ALU} = \mathrm{ALU}(A, B, C),\\
&D = F_{ALU}, && Q^{+} = \begin{cases} 0 & \text{si } \texttt{rst} = 1,\\ D & \text{si } \texttt{rst} = 0 \text{ y } \texttt{L} = 1,\\ Q & \text{en otro caso,} \end{cases}
\end{aligned}
$$

siendo Q^{+} el valor que adopta el registro en cada flanco de subida de `clk` (el reinicio tiene prioridad sobre la carga). De esta ecuación se sigue una consecuencia temporal importante, verificada en simulación (§5): con `L` $= 0$ el registro conserva su valor y `salida` muestra el valor *anterior*, aunque F_{ALU} cambie por efecto de A, B o C. La salida externa sale del registro, no de la ALU.

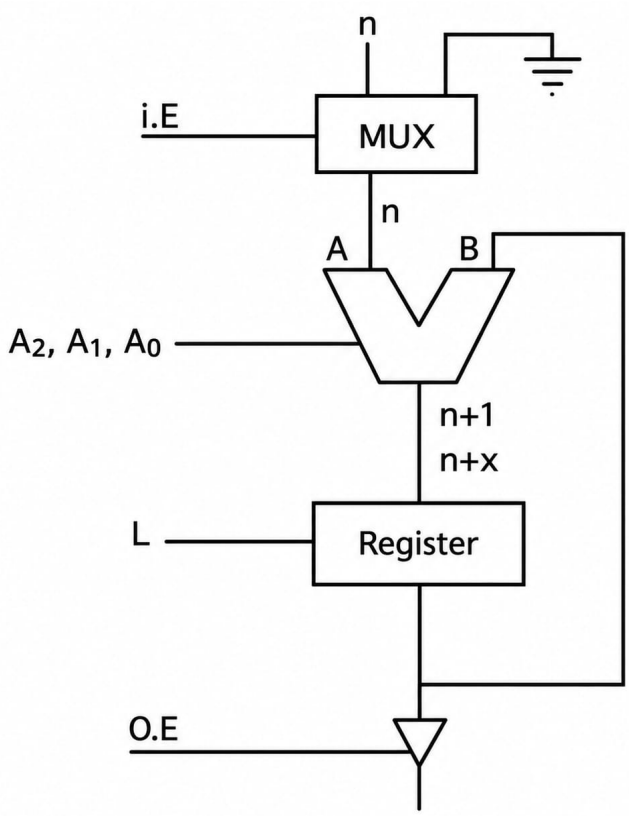


Fig. 1. Diagrama del Datapath 1 proporcionado para la practica: MUX de entrada, ALU, registro con realimentacion hacia B y buffer de salida controlado por O.E.

El reinicio síncrono no aparece en la fotografía, pero es necesaria: sin ella, B no puede garantizarse en cero y la única vía de cargar `dato_ext` en el registro sería sumar con $B = 0$ (§2.4). Se optó por `rst` síncrono con prioridad sobre `L`, patrón ya usado en los registros del resto del proyecto.

2.3 Módulos VHDL

Los modelos siguen las convenciones del proyecto: VHDL-2008 conservador [10], un fichero por entidad, paquetes `ieee.std_logic_1164` y `ieee.numeric_std`, y combinación de los estilos de descripción: *dataflow* para el MUX y la ALU, *comportamental* para el registro y el buffer, y *estructural* para la integración [13]. Todos los módulos nuevos declaran `generic ANCHO : natural := 8`, de modo que el ancho de los buses es parametrizable y los mismos bancos de prueba pueden comprobar anchos distintos [7].

- `mux_entrada`: entradas `dato_ext` y constante cero, selección `sel` (el campo `M` de la palabra de control, con polaridad `'1'` = dato externo, decisión de diseño) y salida `y`, el operando A.
- `alu`: entradas `a`, `b` y código `c` de 3 bits; salida `f`. Un proceso con `case` implementa las tres operaciones de la §2.4 y devuelve cero para los códigos reservados.
- `registro`: entradas `clk`, `rst`, `l` y dato `d`; salida `q`. El proceso de carga con prioridad de reinicio es el siguiente (extracto de `registro.vhd`, código completo en el Apéndice):

Listado 1.1. `registro.vhd` — proceso de carga con prioridad de reinicio

```
process (clk)
begin
    if rising_edge(clk) then
        if rst = '1' then
            contenido <= (others => '0');
        elsif l = '1' then
            contenido <= d;
        end if;
    end if;
end process;
```

- `buffer_salida`: entradas `d_in` y `oe`; salida `y` con asignación condicional: `y <= d_in when oe = '1' else (others => 'Z')`.
- `datapath`: integración *estructural* que instancia los cuatro módulos por componentes —de modo que el análisis del conjunto no depende del orden de los ficheros—, desempaqueta la palabra de control (`cw(5)`=`oe`, `cw(4)`=`l`, `cw(3)`=`m`, `cw(2 downto 0)`=`c`), realimenta `q` al operando `b` y declara además dos salidas de observación, `f_alu` y `q_reg`, que permiten inspeccionar el resultado combinacional y el valor almacenado en los bancos de prueba y en las capturas de ondas sin formar parte de la salida externa.

Tabla 1. Formato de la palabra de control `cw` (6 bits, de MSB a LSB).

Bits	Campo	Acción	Origen
`cw[5]`	`O.E`	habilita la salida (desde Q)	foto (`O.E.`)
`cw[4]`	`L`	carga del registro (flanco de `clk`)	foto (nombre provis.)
`cw[3]`	`M`	MUX: `1`=`dato_ext`, `0`=cero	decisión
`cw[2:0]`	`C2 C1 C0`	operación de la ALU	foto (`c[0:2]`)

Tabla 2. Operaciones de la ALU: expresiones del material del curso y codificación binaria propuesta (provisional) por el equipo.

C ($c_2c_1c_0$)	Nombre	Ecuación	Usa A	Con $A = x$, $B = n$
`000`	suma	$F = A + B$	sí	$n + x$
`001`	doble y suma	$F = (B \ll 1) + A$	sí	$2n + x$
`010`	incremento	$F = B + 1$	no	$n + 1$
`011`–`111`	reservado	$F = 0$ (propuesta)	—	—

2.4 Palabra de control y operaciones de la ALU

La señal `cw` tiene 6 bits y su formato, de más significativo a menos, es `[O.E, L, M, C2, C1, C0]` (Tabla 1). Los campos `O.E`, `L` y el triplete `c[0:2]` están tomados del material del curso; la polaridad de `M` y la correspondencia bit a bit de los campos con `cw` son decisiones de diseño del equipo.

La Tabla 2 resume las operaciones de la ALU. Las *expresiones* ($n+x$, $2n+x$, $n+1$) provienen del material del curso; la *codificación binaria* de la columna C es una propuesta explícita y provisional del equipo, acordada para poder implementar y verificar el diseño: la codificación original del profesor no se pudo leer en la fotografía. Los códigos `011`–`111` quedan reservados, y se les asigna salida cero como valor por defecto inocuo hasta contar con evidencia de su significado real.

La notación n y x del material de clases se corresponde con los operandos del circuito: n es el contenido actual del registro, es decir, el operando B; x es el valor externo presente en `dato_ext` cuando `M` $= 1$, es decir, el operando A tras el MUX. Con esa compatibilidad, las operaciones pedidas se obtienen así: $n + x$ con `000` y `M` $= 1$; $2n + x$ con `001` y `M` $= 1$, usando el desplazamiento lógico a la izquierda de un bit de B *dentro* de la ALU; $n+1$ con `010`, operación que ignora A y por eso hace irrelevante `M`; y la carga directa de x en el registro se consigue con un `rst` previo ($B = 0$) y `000` con `M` $= 1$, de modo que $F = x+0 = x$. Como limitación documentada, con solo estas tres operaciones no es posible cargar A en un registro cuyo B sea distinto de cero: por eso el reinicio es necesario.

2.5 Decisiones de diseño y limitaciones

– **Ancho parametrizable**: la fotografía no permite leer el ancho de los buses; se adopta `ANCHO := 8` como valor por defecto y todos los bancos de prueba trabajan también con anchos distintos (por ejemplo, 4 bits) para demostrar la parametrización.

– **Sin banderas**: no se modelan acarreo ni indicador de cero. Toda la aritmética es módulo $2^{\texttt{ANCHO}}$ y los desbordamientos son silenciosos: con 8 bits, doblar sin pérdida exige $n \leq 127$, y $2n + x$ sin envoltura exige $2n + x \leq 255$. Estos límites se caracterizan empíricamente en la Sección 5.
– **Sin FSM, sin shifter externo**: únicamente los módulos del esquema del curso más `clk` y `rst`.
– **Solo simulación**: todas las fuentes se verificaron *exclusivamente* por simulación con GHDL (backend `mcode`). GHDL dispone de un flujó experimental de síntesis (`ghdl --synth`) [4]; en este proyecto no se *utilizó* ese flujo ni se evaluaron la síntesis ni la implementación física, por lo que no se afirma sintetizabilidad.

3 Propuesta Experimental

La metodología sigue un flujo descendente: de la especificación de la §2 se pasó a los modelos VHDL-2008, después a la integración estructural y, en paralelo, a los bancos de prueba; la verificación se hizo por simulación y el análisis de resultados por inspección de ondas. Los modelos combinan los estilos *dataflow*, *comportamental* y *estructural* [13] y sus procesos siguen las sentencias secuenciales del lenguaje [1], con un estilo de diseño y de entidades consistente con el de los textos de referencia del curso [7,10].

Las herramientas de simulación son GHDL [4], invocado con el estándar `--std=08` y volcado de ondas a formato VCD, y GTKWave [5] para la inspección de las ondas y la exportación de capturas mediante scripts Tcl. La ejecución se apoya en los scripts oficiales del repositorio: `vhdl-analizar` analiza las fuentes, `vhdl-sim` analiza, elabora y ejecuta cada banco de prueba, y `latexbuild` compila este informe en formato Springer (llncs).

Se definieron cinco bancos de prueba autochequeados, uno por módulo y uno de integración:

– `tb_mux_entrada`: comprueba todas las selecciones del MUX con el genérico `ANCHO` igual a 8 y a 4 bits.
– `tb_registro`: verifica el reinicio, la carga con `L` y la retención cuando `L` $= 0$.
– `tb_buffer_salida`: comprueba que con `oe` $= 1$ propaga el dato y con `oe` $= 0$ entrega alta impedancia.
– `tb_alu`: 27 comprobaciones sobre las tres operaciones (incluidos operandos nulos y máximos), los desbordamientos ($200 + 100$ sobre 8 bits, $200 \ll 1$ con pérdida del bit más significativo, $255 + 1$ envolviendo a cero), los códigos reservados y un segundo ejemplo con `ANCHO` igual a 4 bits.
– `tb_datapath`: aplica las cinco palabras de control verificadas, reproduce el ejemplo completo de tres ciclos ($x = 5$ con reinicio inicial) y ejecuta la prueba discriminante `L` $= 0$: con `CW = 100010` la ALU calcula $n + 1$, pero la salida externa debe seguir siendo n.

Cada banco de prueba termina siempre con un mensaje de éxito (`tb_...: OK`) y falla con `assert ... severity failure` ante cualquier desviación respecto al valor esperado. El pulso de reloj se genera dentro del proceso estimulador —patrón de todos los bancos de prueba del proyecto, ya que `vhdl-sim` no impone un límite de tiempo—, de modo que toda simulación termina de forma determinista. Además, la integración continua del repositorio ejecuta la batería completa de `tb_*.vhd` en cada cambio.

El procedimiento de evaluación, repetido para cada experimento, fue: (1) analizar la unidad con el script `vhdl-analizar`; (2) ejecutar el banco de prueba con `vhdl-sim` y comprobar en consola que termina con `OK` y sin errores de GHDL; (3) volcar el VCD resultante; (4) capturar las ondas relevantes a PNG con GTKWave; y (5) contrastar los valores esperados y observados de las señales. Criterio de éxito: los cinco bancos terminan con `OK` y ninguna aserción se dispara. Los casos en que el resultado aritmético difiere del valor matemático exacto por envoltura módulo $2^{\texttt{ANCHO}}$ no se consideran fallos, sino comportamiento documentado en la Sección 2.5 y analizado en la Sección 5.

4 Simulaciones de VHDL

Las capturas que siguen se obtuvieron ejecutando los bancos de prueba con `scripts/vhdl-sim datapath-1 <tb>`, que vuelca las ondas en `vhdl/datapath-1/salidas/<tb>.vcd`, y abriéndolas después en GTKWave mediante un script Tcl que fija el rango de tiempos visible, el orden de las señales y exporta el PNG a `figuras/`. Para que la fuente sea legible, cada figura recorta una ventana corta de la traza; los valores que se leen en las ondas coinciden con las aserciones del banco de prueba correspondiente.

La Fig. 2 muestra el tramo principal de la simulación del datapath completo (5–95 ns), con las señales `clk`, `rst`, `dato_ext`, `cw`, `f_alu`, `q_reg` y `salida`. En el primer ciclo (tras el reinicio de la traza, a $t \approx 10$ ns) la palabra de control `111000` carga $n = 5$: `f_alu` pasa a `0x05` de forma combinacional y, en el flanco, `q_reg` y `salida` se actualizan a `0x05`. En el segundo ciclo, `111001` ($2n + x$) lleva `f_alu` a `0x0F` y el registro a `0x0F`. El tramo central (≈50–65 ns) es la prueba discriminante: con `CW=100010` ($L = 0$, $n + 1$) la ALU calcula `f_alu=0x10`, pero `q_reg` y `salida` permanecen en `0x0F` incluso después del flanco, porque el registro no carga.

La salida del buffer en tramo de alta impedancia queda en la Fig. 3, que recorta la ventana final de la misma simulación (105–170 ns). Al cargar `dato_ext=0x3C` con `CW=011000` (`OE` = 0), `salida` está en estado Z; la palabra de control `100000` habilita el buffer y muestra el valor guardado; y, por último, `n+1` con `CW=110010` lleva el registro a `0x3D`, visible bajo `q_reg`, `f_alu` y `salida`. Esta ventana es la contraprueba del capítulo de arquitectura: la carga con `L`=1 y la baja a alta impedancia con `OE` = 0 confirman que el registro y el buffer separan el almacenamiento del *drive*.

La Fig. 4 presenta la simulación de la ALU en su instancia de 8 bits (95–170 ns), con las señales `a8`, `b8`, `c` y `f8`. Se ve el barrido de las tres operaciones

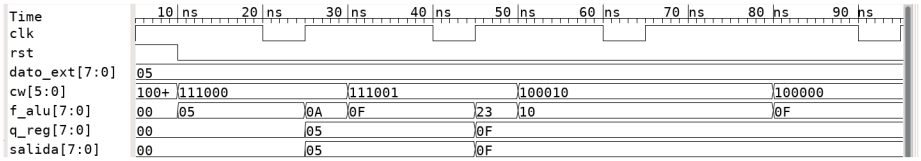


Fig. 2. Simulación del datapath completo (`tb_datapath`), ventana de 5–95 ns. Se aprecia la actualización de `q_reg` y `salida` solo en los flancos con `L`=1 y, en el tramo central, que con `L`=0 la salida no sigue a `f_alu`: la salida proviene del registro.

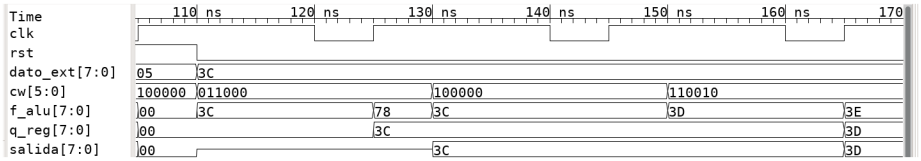


Fig. 3. Simulación del datapath completo (`tb_datapath`), ventana de 105–170 ns. Con `CW=011000` la salida está en alta impedancia (Z); `100000` restaura el buffer con el valor guardado y `110010` ($n+1$) lo actualiza a `0x3D`.

—suma (`c=000`), doble y suma (`c=001`) e incremento (`c=010`)— y los tres desbordamientos documentados en la Sección 2.5: `100+200=0x2C`, `200`$\ll$`1=0x90` y `255+1=0x00`, todos por envoltura módulo 2^8. Los cinco códigos reservados del barrido y la instancia de 4 bits devuelven `0`; su formulario está tabulado en la Tabla 4.

Por último, la Fig. 5 recoge la ventana de la instancia de 4 bits de la misma simulación (215–250 ns), en la que se verifica que el modelo es genérico respecto al parámetro `ANCHO`: `3+1=4`, `9`$\ll$`1=2` (18 mod 16) y `15+1=0`. Las demás capturas —`tb_mux_entrada` y `tb_buffer_salida`— confirman el comportamiento combinacional de los bloques restantes y no se muestran por límite de espacio.

5 Resultados

5.1 Ejecución de los bancos de prueba

Los cinco bancos de prueba se ejecutaron con el script `vhdl-sim` del repositorio y todos terminaron con el mensaje de éxito, sin que se disparase ninguna aserción (Tabla 3). El instante final es el del `report ... OK`, que depende del número de estímulos de cada banco y no de un tiempo de simulación impuesto por la herramienta, ya que el reloj se genera dentro del propio proceso estimulador.

También se verificó el análisis de todas las fuentes y bancos de prueba con `scripts/vhdl-analizar datapath-1` (cinco fuentes y cinco bancos), sin mensajes de error de GHDL 4.1.0 con el estándar `--std=08`. La integración continua del repositorio reproduce esta misma batería.

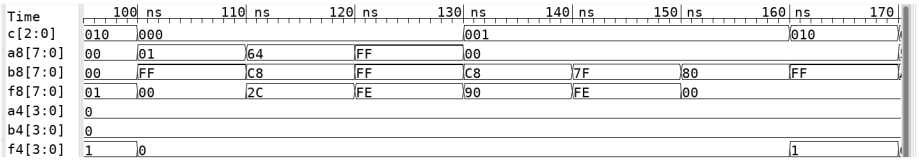


Fig. 4. Simulación de la ALU (`tb_alu`) en 8 bits (`a8`, `b8`, `f8`), ventana de 95–170 ns. El código `c` recorre las tres operaciones y los tres desbordamientos, que envuelven módulo 2^8.

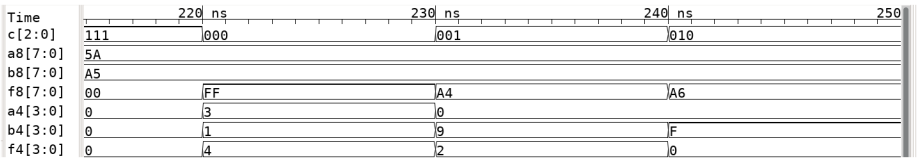


Fig. 5. Simulación de la ALU (`tb_alu`) en 4 bits (`a4`, `b4`, `f4`), ventana de 215–250 ns. Las mismas tres operaciones en `ANCHO`=4: `3+1=4`, `9≪1=2` y `15+1=0` muestran la envoltura módulo 2^4.

5.2 Operaciones de la ALU

La Tabla 4 recoge, para una muestra representativa de los 27 casos comprobados, el valor de los operandos, el código y el resultado observado, con el resultado matemático de referencia. Los tres desbordamientos documentados en la Sección 2.5 se manifiestan como desbordamiento modular (envoltura) de módulo 2^8: $200 + 100$ da `0x2C` ($= 44$), porque $300 \bmod 256 = 44$; doblar 200 da `0x90` ($= 144$), porque $400 \bmod 256 = 144$; y $255 + 1$ da `0x00`. Ninguno de estos casos es un fallo del modelo, sino la consecuencia de trabajar sin banderas de acarreo ni de desbordamiento.

La instancia con `ANCHO`=4 (filas inferiores de la Tabla 4) confirma que el *generic* parametriza la ruta aritmética completa y no solo los anchos de los puertos. Las tres operaciones se repiten con operandos de 4 bits y los límites se desplazan de $2^8 - 1 = 255$ a $2^4 - 1 = 15$: $3+1 = 4$ cabe sin envolver, $2 \cdot 9 = 18 \equiv 2 \pmod{16}$ y $15 + 1 = 0$. El resultado observado coincide en los tres casos con la referencia matemática, lo que valida la parametrización del diseño sin cambios de descripción (es la misma entidad `alu` instanciada con otro valor del *generic*).

5.3 Palabras de control en el datapath

La Tabla 5 contrasta, para cada palabra de control empleada, el resultado esperado con el observado en `tb_datapath`. Las filas de los ciclos 1 y 2 corresponden al ejemplo de tres ciclos de la Sección 2.4; la prueba discriminante (`CW=100010`) confirma que con `L`=0 la salida externa conserva el valor del registro y no adopta el nuevo resultado de la ALU. La carga con `CW=011000` muestra además la alta impedancia cuando `O.E` $= 0$.

Tabla 3. Resultado de la ejecución de los cinco bancos de prueba (`scripts/vhdl-sim datapath-1 <tb>`).

Banco de prueba	Módulo bajo prueba	Fin	Resultado
`tb_mux_entrada`	`mux_entrada`	60 ns	OK
`tb_registro`	`registro`	50 ns	OK
`tb_buffer_salida`	`buffer_salida`	40 ns	OK
`tb_alu`	`alu`	250 ns	OK (27 aserciones)
`tb_datapath`	`datapath`	170 ns	OK

Tabla 4. Resultados de la ALU observados en `tb_alu` (extracto de los 27 casos; 8 bits salvo indicación).

C	A	$B = n$	F observado	Referencia	Observación
`000`	`0A`	`05`	`0F`	$5 + 10 = 15$	$n + x$
`000`	`3C`	`00`	`3C`	$x + 0$	carga de x
`001`	`0A`	`05`	`14`	$2 \cdot 5 + 10 = 20$	$2n + x$
`001`	`00`	`7F`	`FE`	$2 \cdot 127 = 254$	límite exacto
`010`	`A5`	`05`	`06`	$5 + 1 = 6$	ignora A
`000`	`64`	`C8`	`2C`	300 mod 256	**envoltura** ($= 44$)
`001`	`00`	`C8`	`90`	400 mod 256	**envoltura** ($= 144$)
`010`	`00`	`FF`	`00`	256 mod 256	**envoltura**
`011`–`111`	`5A`	`A5`	`00`	reservado	valor por defecto
Instancia con `ANCHO`*=4:*					
`000`	`0011`	`0001`	`0100`	$3 + 1 = 4$	$n + x$
`001`	`0000`	`1001`	`0010`	18 mod 16	$2n + x$ ($= 2$)
`010`	`0000`	`1111`	`0000`	16 mod 16	**envoltura**

En todos los casos el valor observado de F_{ALU} y el del registro tras el flanco coinciden con el esperado. El interés de la fila `100010` es que el registro y la ALU divergen deliberadamente: $F_{ALU} =$ `0x10`, pero $Q =$ `salida` $=$ `0x0F`, lo que valida la corrección de diseño de la §2.2 (la salida sale del registro).

5.4 Comportamiento temporal

Del análisis de las ondas (Fig. 2) se desprenden dos observaciones. Primero, la latencia del resultado es de un flanco: cuando `L`=1, el valor nuevo de F_{ALU} está disponible de forma combinacional en el mismo ciclo, pero solo se hace visible en `salida` tras el flanco de subida, porque atraviesa el registro. Segundo, la salida combinacional de la ALU (F_{ALU}) sí puede cambiar en cualquier momento del ciclo al variar A, B o C; por eso se expone como salida de observación `f_alu`, separada de la salida externa. El diseño resulta así coherente con la separación entre la parte combinacional y la secuencial, y con el modelo de la §2.2.

Las ventanas dedicadas a la ALU refuerzan la lectura combinacional. En la Fig. 4, `f8` responde a cada cambio de `c8` o de los operandos en el instante exacto del cambio, sin esperar flanco de reloj alguno: la suma $100 + 200$ produce `0x2C`, doblar 200 produce `0x90` y el caso límite $255 + 1$ produce `0x00` en su

Tabla 5. Resultados del datapath observados en `tb_datapath` (valores en hexadecimal).

CW	F_{ALU}		Q		salida	Observación
	esp.	*obs.*	*esp.*	*obs.*		
111000	05	05	05	05	05	$n + x$ guardado
111001	0F	0F	0F	0F	0F	$2n + x$ guardado
100010	10	10	0F	0F	0F	$L = 0$: salida = registro
100000	0F	0F	0F	0F	0F	conservar y mostrar
011000	3C	3C	3C	3C	Z	carga con O.E = 0
110010	3D	3D	3D	3D	3D	$n + 1$ guardado

correspondiente ventana. Por su parte, la Fig. 5 recoge el mismo banco con `ANCHO`=4 en el tramo final: `c4` recorre el código reservado (`111`, con `f4`=0) y después las tres operaciones de la Tabla 4 ($3 + 1 = 4$; $2 \cdot 9 \equiv 2 \pmod{16}$; $15 + 1 = 0$), mientras la instancia de 8 bits sigue estimulada en paralelo con sus propios operandos (`f8` responde a sus estímulos en las mismas ventanas). Ambas ventanas ilustran, además, un rasgo del entorno de simulación: la regla temporal en nanosegundos y el escalado automático (*zoom*) dependen de la herramienta, no del modelo VHDL de `vhdl/datapath-1`.

En cuanto a coste, y sin entrar en síntesis (el flujo es solo simulación), el datapath resuelve cada micro-operación en un único ciclo de reloj, salvo la secuencia de inicialización que requiere un `rst` previo para poder cargar `dato_ext` con $B = 0$. No hay por tanto operaciones que abarquen varios ciclos ni *overhead* de una máquina de estados, a cambio de que la secuencia la deba aportar el entorno externo; desde la perspectiva de las métricas de rendimiento clásicas, el tiempo de ejecución de una secuencia es proporcional al número de palabras de control aplicadas [17,8].

5.5 Discusión

Los resultados confirman el funcionamiento previsto y, al mismo tiempo, delimitan las limitaciones que ya se anticiparon en el diseño. La más importante es la aritmética silenciosa: al no haber banderas, el modelo no distingue un resultado correcto de uno envuelto, de modo que la validez de $2n + x$ y de $n + x$ queda condicionada a los rangos indicados en la Sección 2.5 ($n \leq 127$ para doblar sin pérdida, y $2n + x \leq 255$ en 8 bits). La segunda es la imposibilidad de cargar A sobre un registro con $B \neq 0$ usando solo las tres operaciones pedidas; se resuelve con el reinicio, que es una decisión de diseño adicional y no un elemento del esquema original. Por último, la codificación binaria de la ALU es provisional: el comportamiento del modelo es correcto *bajo esa codificación*, pero si el material del curso fija otra, habría que reajustar la tabla y repetir la verificación. Los bancos de prueba están escritos de forma que ese cambio afectaría solo a la tabla de la §2.4 y a los estímulos de `tb_alu` y `tb_datapath`.

6 Conclusiones

Se diseñó y verificó por simulación el Datapath 1: una ruta de datos de 8 bits parametrizable, compuesta por un MUX $2 \rightarrow 1$, una ALU de tres operaciones, un registro con carga y un buffer tri-estado de salida, integrados de forma estructural y realimentados del registro hacia la ALU. Los cinco objetivos planteados en la introducción se cumplieron: los módulos se describieron en VHDL-2008 con ancho parametrizable, se integraron en una entidad estructural, se verificaron con cinco bancos de prueba autochequeados y se analizó su comportamiento temporal a partir de las ondas de GTKWave.

El resultado principal es una ruta de datos correcta bajo el modelo especificado: las tres operaciones ($n + x$, $2n + x$ y $n + 1$), la carga del registro, la retención con `L`=0 y la alta impedancia con `O.E` $= 0$ se comportan como se esperaba, incluidos los casos de desbordamiento documentados. La decisión de diseño que separa el resultado combinacional de la ALU (F_{ALU}) del valor almacenado (Q) quedó validada por la prueba discriminante con `CW=100010`: con la carga deshabilitada, la salida externa no sigue a la ALU.

Las principales limitaciones son tres y quedan documentadas: los desbordamientos son silenciosos al no modelarse banderas; solo las tres operaciones pedidas están implementadas, de modo que cargar el operando externo sobre un registro no nulo exige un reinicio previo; y la codificación binaria de la ALU y el formato exacto de la palabra de control son una propuesta del equipo mientras no se disponga de la codificación original. El modelo es, además, una descripción para simulación: los códigos se verificaron *únicamente* por simulación; el flujo de síntesis experimental de GHDL (`ghdl --synth`) no se empleó y la sintetizabilidad y la implementación física no se han evaluado.

Como trabajo futuro inmediato cabe refactorizar tan pronto como se disponga de la codificación real del curso (una rama por cambio, según el flujo del repositorio), añadir banderas de acarreo y de cero preservando la realimentación, y dotar al datapath de una unidad de control mínima —una máquina de estados de estilo Moore o Mealy— que genere la secuencia de palabras de control que aquí aplica el banco de prueba. Una unidad de control así ampliaría el funcionamiento del datapath (secuen- ciar microoperaciones sin seguir aplicando `cw` a mano), aunque por sí sola no bastaría para constituir un procesador completo: faltarían elementos propios de un procesador, como una memoria de instrucciones, un contador de programa y formateo/carga de la unidad de datos.

Referencias

1. Ashenden, P.J.: The Designer's Guide to VHDL. Morgan Kaufmann (Elsevier), San Francisco, 3rd edn. (2008), cap. 9 – sentencias secuenciales y procesos (wait, if/case), usados en el modelado sincrónico del registro con carga; primera guía completa de VHDL-2008 (prefacio); ISBN: 978-0-12-088785-9
2. Brown, S.D., Vranesic, Z.G.: Fundamentals of Digital Logic with VHDL Design. McGraw-Hill, New York, 3rd edn. (2009), capítulos de diseño de FSM con VHDL –

ejemplos de codificación de máquinas de estado y bancos de registros usados como referencia de estilo; ISBN: 978-0-07-352953-0

3. Bryant, R.E., O'Hallaron, D.R.: Computer Systems: A Programmer's Perspective. Pearson, Boston, 3rd edn. (2015), cap. 4 (§4.1–4.4) – datapath de ciclo único: organización de la ruta de datos y de las señales que la gobernan; ISBN: 978-0-13-409266-9
4. GHDL developers: GHDL: the VHDL simulator. https://ghdl.github.io/ghdl/ (2026), secciones "Overview" y "Usage" (opciones –std=08 y –vcd); consulta: 2026-10-19
5. GTKWave developers: GTKWave analyzer. https://gtkwave.sourceforge.net/ (2026), sección "Documentation" y guía de scripting Tcl (addSignalsFromList, Grab_To_File), usadas para las capturas de este informe; consulta: 2026-10-19
6. Hamacher, V.C., Vranesic, Z.G., Zaky, S.G.: Computer Organization. McGraw-Hill, New York, 5th edn. (2002), cap. 1 – estructura básica de un computador: CPU compuesta por ruta de datos (datapath) y unidad de control, marco conceptual del informe; ISBN: 978-0-07-232086-2
7. Harris, S.L., Harris, D.M.: Digital Design and Computer Architecture: ARM Edition. Morgan Kaufmann (Elsevier), San Francisco, 1st edn. (2015), cap. 4 – lenguajes de descripción de hardware con secciones de VHDL, referencia de estilo para entidades, procesos y bancos de prueba; ISBN: 978-0-12-800056-4
8. Hennessy, J.L., Patterson, D.A.: Computer Architecture: A Quantitative Approach. Morgan Kaufmann (Elsevier), San Francisco, 6th edn. (2017), cap. 1 (§1.1–1.2) – principios cuantitativos de diseño y métricas de coste/rendimiento; ISBN: 978-0-12-811905-1
9. IEEE: IEEE std 1164-1993 – IEEE standard multivalue logic system for VHDL model interoperability (Std_logic_1164). Tech. rep., Institute of Electrical and Electronics Engineers (1993). https://doi.org/10.1109/IEEESTD.1993.115571, paquete std_logic_1164: lógica resoluble de 9 valores usada en todos los puertos del diseño (incluida la alta impedancia del buffer); IEEE Xplore doc. 257627; DOI: 10.1109/IEEESTD.1993.115571 (verificado en Crossref y IEEE Xplore el 2026-10-20)
10. IEEE: IEEE std 1076-2008 – IEEE standard VHDL language reference manual. Tech. rep., Institute of Electrical and Electronics Engineers (2009). https://doi.org/10.1109/IEEESTD.2011.5967868, definición del lenguaje VHDL-2008 implementado en los modelos (adopción ISO/IEC 61691-1-1); DOI: 10.1109/IEEESTD.2011.5967868
11. Katz, R.H., Borriello, G.: Contemporary Logic Design. Pearson Prentice Hall, Upper Saddle River, NJ, 2nd edn. (2005), capítulos de diseño secuencial – desglose de la señal de control en campos codificados, análogo al reparto de nuestra palabra CW en O.E, L, M y C; ISBN: 978-0-201-30857-0
12. Mano, M.M., Kime, C.R., Martin, T.: Logic and Computer Design Fundamentals. Pearson, Upper Saddle River, NJ, 5th edn. (2016), cap. 6 – circuitos secuenciales: registros con habilitación de carga y reloj sincrónico, base del modelado del registro del datapath; ISBN: 978-0-13-3760637
13. Navabi, Z.: VHDL: Analysis and Modeling of Digital Systems. McGraw-Hill, New York, 2nd edn. (1998), capítulos de modelado – estilos de descripción (dataflow, behavioral, structural) combinados en la jerarquía estructural del datapath; ISBN: 978-0-07-046479-7

14. Patterson, D.A., Hennessy, J.L.: Computer Organization and Design: The Hardware/Software Interface (RISC-V Edition). Morgan Kaufmann (Elsevier), Amsterdam, 1st edn. (2017), cap. 1 (§1.1) – abstracción por capas del hardware; cap. 4 (§4.3–4.4) – separación datapath / unidad de control en una implementación de ciclo único, marco conceptual de la arquitectura descrita; ISBN: 978-0-12-812275-4 (1. a ed. de la versión RISC-V; ISBN verificado en Elsevier y listas de curso el 2026-10-20)
15. Rabaey, J.M., Chandrakasan, A., Nikolić, B.: Digital Integrated Circuits: A Design Perspective. Pearson Prentice Hall, Upper Saddle River, NJ, 2nd edn. (2003), cap. 11 (§11.1–11.4) – datapaths y ALU: sumadores, corrimientos y estructuras barrel, aplicado al desplazamiento interno que calcula 2n dentro de la ALU; ISBN: 978-0-13-090996-1
16. Roth, C.H., Kinney, L.L.: Fundamentals of Logic Design. Cengage Learning, Stamford, CT, 7th edn. (2013), unidades de máquinas de estado finito – tablas de transición y diagramas de estados, tomadas como contrapunto de la ruta sin FSM de este diseño; ISBN: 978-1-133-62847-7
17. Stallings, W.: Computer Organization and Architecture: Designing for Performance. Pearson, Upper Saddle River, NJ, 10th edn. (2015), cap. 2 – métricas de rendimiento (tiempo de ejecución y ciclos); ISBN: 978-0-13-410161-3
18. Wakerly, J.F.: Digital Design Principles and Practices. Prentice Hall (Pearson), Upper Saddle River, NJ, 4th edn. (2005), sección de buffers tri-estado – salida en alta impedancia (Z) para compartir un bus, aplicada al buffer de salida con O.E.; ISBN: 978-0-13-186389-7

A Código VHDL

Se reproduce a continuación el código completo de los módulos de la unidad que se han verificado en este informe. Se inserta desde las copias de presentación guardadas en `apendice/`, con el código idéntico al VHDL funcional de `vhdl/datapath-1/src/` (misma lógica, mismos recursos y mismos puertos) salvo los saltos de línea, ajustados a límites sintácticos para que las instrucciones se puedan leer y copiar sin ambigüedad, y los comentarios, reformateados al mismo ancho y redactados en términos técnicos neutros.

Listado 1.2. `mux_entrada.vhd` — MUX 2 → 1 de entrada

```
-- ======================================================
-- MUX de entrada del Datapath 1 (2 -> 1)
-- Elige el operando A de la ALU: constante 0 (sel = '0')
-- o el valor externo dato_ext (sel = '1').
-- sel: decisión de diseño provisional
--      (polaridad modificable).
-- dato_ext: nombre provisional
--      (la etiqueta del esquema no se leyó).
-- ======================================================
library ieee;
use ieee.std_logic_1164.all;

entity mux_entrada is
    generic (
```

```
        ANCHO : natural := 8
    );
    port (
        dato_ext : in  std_logic_vector(ANCHO - 1 downto 0);
        sel      : in  std_logic;
        y        : out std_logic_vector(ANCHO - 1 downto 0)
    );
end entity;

architecture comportamental of mux_entrada is
begin
    y <= dato_ext when sel = '1' else (others => '0');
end architecture;
```

Listado 1.3. `alu.vhd` — ALU con las tres operaciones y códigos reservados a cero

```
-- ======================================================
-- ALU del Datapath 1
-- Especificación de implementación propuesta por el
-- equipo; los códigos NO están confirmados como
-- provenientes del material oficial del curso.
-- Operaciones provisionales acordadas:
--   C=000: F = A+B        (n+x con M=1; n con M=0)
--   C=001: F = (B<<1)+A (2n+x; desplazamiento interno,
--                        sin shifter)
--   C=010: F = B+1        (n+1, ignora A)
--   C=011..111: reservado, F = 0 (provisional,
--                        pendiente de evidencia)
-- Combinacional, sin signo, módulo 2^ANCHO. Sin
-- banderas: los desbordamientos son silenciosos
-- (limitación documentada).
-- ======================================================
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity alu is
    generic (
        ANCHO : natural := 8
    );
    port (
        a : in  std_logic_vector(ANCHO - 1 downto 0);
        b : in  std_logic_vector(ANCHO - 1 downto 0);
        c : in  std_logic_vector(2 downto 0);
        f : out std_logic_vector(ANCHO - 1 downto 0)
    );
end entity;

architecture comportamental of alu is
begin
    process (a, b, c)
    begin
```

```
        case c is
            when "000" =>
                -- suma: n+x (A=x) o n (A=0);
                -- con B=0 y A=x sirve para cargar
                f <= std_logic_vector(
                          unsigned(a) + unsigned(b));
            when "001" =>
                -- desplazamiento lógico de B a la
                -- izquierda y suma: 2n+x
                f <= std_logic_vector(
                    shift_left(unsigned(b), 1)
                        + unsigned(a));
            when "010" =>
                -- incremento de B: n+1 (A no participa)
                f <= std_logic_vector(unsigned(b) + 1);
            when others =>
                -- reservado (011..111): F = 0 provisional
                f <= (others => '0');
        end case;
    end process;
end architecture;
```

Listado 1.4. `registro.vhd` — registro con carga y reinicio síncrono

```
-- ======================================================
-- Registro de datos del Datapath 1
-- Almacena n (operando B de la ALU) y lo realimenta a la
-- ALU. Carga habilitada con l: nombre provisional tomado
-- de la etiqueta «L» del esquema del curso. rst es
-- decisión de diseño adicional: no aparece en el esquema
-- y se añade para inicializar B = 0 (necesario para
-- cargar dato_ext con op = 000). Reset síncrono con
-- prioridad sobre l. La lógica equivale a un registro
-- con habilitación y reinicio síncrono estándar.
-- ======================================================
library ieee;
use ieee.std_logic_1164.all;

entity registro is
    generic (
        ANCHO : natural := 8
    );
    port (
        clk : in  std_logic;
        rst : in  std_logic;
        l   : in  std_logic;
        d   : in  std_logic_vector(ANCHO - 1 downto 0);
        q   : out std_logic_vector(ANCHO - 1 downto 0)
    );
end entity;

architecture comportamental of registro is
```

```
    signal contenido : std_logic_vector(ANCHO - 1 downto 0)
        := (others => '0');
begin
    process (clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                contenido <= (others => '0');
            elsif l = '1' then
                contenido <= d;
            end if;
        end if;
    end process;

    q <= contenido;
end architecture;
```

Listado 1.5. `buffer_salida.vhd` — buffer tri-estado con habilitación

```
-- ======================================================
-- Buffer tri-estado de salida del Datapath 1
-- Etiqueta del esquema del curso: «O.E.» (Output Enable).
-- oe = '1' saca el dato; oe = '0' deja la salida en alta
-- impedancia. Nombre del puerto: decisión de diseño (el
-- esquema solo muestra la etiqueta O.E.).
-- ======================================================
library ieee;
use ieee.std_logic_1164.all;

entity buffer_salida is
    generic (
        ANCHO : natural := 8
    );
    port (
        d_in : in  std_logic_vector(ANCHO - 1 downto 0);
        oe   : in  std_logic;
        y    : out std_logic_vector(ANCHO - 1 downto 0)
    );
end entity;

architecture comportamental of buffer_salida is
begin
    y <= d_in when oe = '1' else (others => 'Z');
end architecture;
```

Listado 1.6. `datapath.vhd` — integración estructural, desempaquetado de `cw` y realimentación

```
-- ======================================================
-- Datapath 1: integración estructural del diagrama
-- MUX de entrada -> ALU -> Registro -> Buffer de salida,
-- con Q realimentada al operando B de la ALU.
```

```
-- Sin FSM y sin shifter externo (el desplazamiento de
-- 2n+x vive dentro de la ALU).
--
-- CW = [O.E, L, M, C2, C1, C0]:
--   cw(5) = O.E: habilita la salida
--           (desde Q, nunca desde F)
--   cw(4) = L:   carga F_ALU en el registro
--           en el flanco de clk
--   cw(3) = M:   '1' => dato_ext, '0' => constante cero
--   cw(2..0) = C: operación de la ALU
-- Especificación de implementación propuesta por el
-- equipo; los códigos NO están confirmados como
-- provenientes del profesor.
--
-- Se usa instanciación por componentes para que el
-- análisis sea independiente del orden
-- de los ficheros (scripts/vhdl-analizar).
--
-- f_alu y q_reg son salidas de observación para
-- testbenches y capturas de ondas.
-- ======================================================
library ieee;
use ieee.std_logic_1164.all;

entity datapath is
    generic (
        ANCHO : natural := 8
    );
    port (
        clk    : in  std_logic;
        rst    : in  std_logic;
        dato_ext : in std_logic_vector(ANCHO - 1 downto 0);
        cw     : in  std_logic_vector(5 downto 0);
        salida : out std_logic_vector(ANCHO - 1 downto 0);
        f_alu  : out std_logic_vector(ANCHO - 1 downto 0);
        q_reg  : out std_logic_vector(ANCHO - 1 downto 0)
    );
end entity;

architecture estructural of datapath is
    component mux_entrada is
        generic (
            ANCHO : natural := 8
        );
        port (
            dato_ext : in std_logic_vector(ANCHO - 1
                downto 0);
            sel : in  std_logic;
            y   : out std_logic_vector(ANCHO - 1 downto 0)
        );
    end component;
```

```
    component alu is
        generic (
            ANCHO : natural := 8
        );
        port (
            a : in  std_logic_vector(ANCHO - 1 downto 0);
            b : in  std_logic_vector(ANCHO - 1 downto 0);
            c : in  std_logic_vector(2 downto 0);
            f : out std_logic_vector(ANCHO - 1 downto 0)
        );
    end component;

    component registro is
        generic (
            ANCHO : natural := 8
        );
        port (
            clk : in  std_logic;
            rst : in  std_logic;
            l   : in  std_logic;
            d   : in  std_logic_vector(ANCHO - 1 downto 0);
            q   : out std_logic_vector(ANCHO - 1 downto 0)
        );
    end component;

    component buffer_salida is
        generic (
            ANCHO : natural := 8
        );
        port (
            d_in : in  std_logic_vector(ANCHO - 1 downto 0);
            oe   : in  std_logic;
            y    : out std_logic_vector(ANCHO - 1 downto 0)
        );
    end component;

    -- desempaquetado de la palabra de control
    signal oe : std_logic;
    signal l  : std_logic;
    signal m  : std_logic;
    signal c  : std_logic_vector(2 downto 0);

    -- buses internos (replica del diagrama):
    --   a_bus = MUX -> ALU
    --   b_bus = Q -> ALU
    --   f_bus = ALU -> D del registro
    --   q_bus = salida del registro
    signal a_bus : std_logic_vector(ANCHO - 1 downto 0);
    signal b_bus : std_logic_vector(ANCHO - 1 downto 0);
    signal f_bus : std_logic_vector(ANCHO - 1 downto 0);
    signal q_bus : std_logic_vector(ANCHO - 1 downto 0);
begin
```

```
    oe <= cw(5);
    l  <= cw(4);
    m  <= cw(3);
    c  <= cw(2 downto 0);

    -- realimentación: Q es el operando B (n)
    b_bus <= q_bus;

    u_mux : mux_entrada
        generic map (ANCHO => ANCHO)
        port map (
            dato_ext => dato_ext,
            sel      => m,
            y        => a_bus
        );

    u_alu : alu
        generic map (ANCHO => ANCHO)
        port map (
            a => a_bus,
            b => b_bus,
            c => c,
            f => f_bus
        );

    u_registro : registro
        generic map (ANCHO => ANCHO)
        port map (
            clk => clk,
            rst => rst,
            l   => l,
            d   => f_bus,
            q   => q_bus
        );

    u_buffer : buffer_salida
        generic map (ANCHO => ANCHO)
        port map (
            -- la salida sale del REGISTRO, no de la ALU
            d_in => q_bus,
            oe   => oe,
            y    => salida
        );

    -- salidas de observación (testbenches y capturas)
    f_alu <= f_bus;
    q_reg <= q_bus;
end architecture;
```